

PODATKOVNE BAZE I

Informatika in podatkovne tehnologije (IPT), Telekomunikacije (TK)
Univerzitetni študijski program (UN)

1. letnik

Ponovimo...

- Kakšni sta najvišja, najnižja in povprečna vrednost izdanih računov?
- Izpišimo id-je računov, ki vsebujejo postavke s količino večjo ali enako 2.
- Izpišite vse cene izdelkov in jih sortirajte padajoče.
- **Vaše** povpraševanje?

Funkcije nad vrednostmi

Funkcije nad vrednostmi

- Funkcije nad **vrednostmi** so tiste funkcije, ki procesirajo **nad posameznimi vrednostmi izbranih stolpcev**.
 - Za razliko od funkcij nad množicami, ki procesirajo **nad vrednostmi celotne tabele**.
- Kombiniramo jih z drugimi stolpci v **SELECT** delu ukaza.
- Podobne funkcijam v programskih jeziki.
- Vrnejo lahko:
 - **Numerične**
 - **Znakovne**
 - **Datumske** in **časovne** vrednosti.

Numerične funkcije

- Funkcije nad **vrednostmi** so tiste funkcije, ki procesirajo **nad posameznimi vrednostmi izbranih stolpcev**.
 - Za razliko od funkcij nad množicami, ki procesirajo **nad vrednostmi celotne tabele**.
- Kombiniramo jih z drugimi stolpci v **SELECT** delu ukaza.
- Podobne funkcijam v programskih jezikih.
- Vrnejo lahko:
 - **Numerične**
 - **Znakovne**
 - **Datumske** in **časovne** vrednosti.

Numerične funkcije

- **Numerične funkcije** vrnejo numerični rezultat, čeprav je lahko argument drugačnega podatkovnega tipa.
- **Primeri** numeričnih funkcij:
 - `ABS()` – Funkcija kot rezultat vrne absolutno vrednost števila.
 - `PI()` – Funkcija vrne vrednost π .
 - `ROUND()` – Funkcija zaokroži število navzgor.
 - `SQRT()` – Funkcija vrne kvadratni koren.
 - `TRUNCATE()` – Funkcija skrajša na določeno število decimalnih mest.
 - ...

Pazimo na argumente!

<https://www.postgresql.org/docs/18/functions-math.html>

Znakovne funkcije

- **Znakovne funkcije** vrnejo kot rezultat vrednost znakovnega tipa.
- **Primeri** znakovnih funkcij:
 - `BIT_LENGTH()` – Funkcija vrne dolžina argumenta v bitih.
 - `CHAR_LENGTH()` – Funkcija vrne število znakov v argumentu.
 - `FORMAT()` – Funkcija vrne število, oblikovano na določeno število decimalnih mest.
 - `REPEAT()` – Funkcija ponovi niz.
 - `REPLACE()` – Funkcija zamenja pojavitve podanih nizov.
 - `SUBSTRING()` – Vrne odrezan niz.
 - `UPPER()` – Funkcija pretvori v velike črke.
 - ...

Pazimo na argumente!

<https://www.postgresql.org/docs/18/functions-string.html>

Datumske in časovne funkcije

- **Datumske in časovne funkcije** vrnejo kot rezultat vrednost datumskega in časovnega tipa.
- **Primeri** znakovnih funkcij:
 - `CURRENT_DATE` – Funkcija vrne trenutni datum.
 - `CURRENT_TIME` – Funkcija vrne trenutni čas.
 - `DATE_FORMAT()` – Funkcija formatira datum.
 - `EXTRACT()` – Vrne želen del datuma/časa.

<https://www.postgresql.org/docs/18/functions-datetime.html>

Pazimo na argumente!

Funkcije nad vrednostmi

- Vse to so zgolj funkcije.
 - Kako jih **smiselno** uporabiti?
- Primeri iz predavanj:
 - *Zaokrožite vse cene rastlin na 1 decimalko.*
 - *Zaokrožite povprečno vrednost računov na eno decimalko.*
 - *Izpišite imena vseh naših zasebnih kupcev z velikimi črkami.*
 - *Koliko dni je minilo od vsakega izdanega računa?*

Funkcije nad vrednostmi

- Vse to so zgolj funkcije.
 - Kako jih **smiselno** uporabiti?
- Primeri iz predavanj:
 - *Zaokrožite vse cene rastlin na 1 decimalko.*
 - *Zaokrožite povprečno vrednost računov na eno decimalko.*
 - *Izpišite imena vseh naših zasebnih kupcev z velikimi črkami.*
 - *Koliko dni je minilo od vsakega izdanega računa?*

Izrazi z uporabo vrednosti

- SQL poizvedbe lahko pišemo tako, da med izbiranjem (SELECT) izvajamo določene **operacije** nad **statičnimi vrednostmi**, **vrednostmi** iz **stolpcev** tabele ter **rezultati funkcij nad množicami**.
 - Tako manipuliramo končni rezultat poizvedbe.
- Operacije, ki jih lahko uporabljamo so npr. **seštevanje**, **odštevanje**, **deljenje**, **množenje**, **lepljenje znakovnih nizov** itd.
- Izvajamo lahko kombinacijo operacij in kombinacijo vrst vrednosti.

Izrazi z uporabo vrednosti

- Lahko na primer:
 - Seštejemo
 - Datum računa + 10 let
 - Odštejemo
 - Cena - popust
 - Delimo
 - Stopnja DDV /100
 - Množimo
 - Količina * cena
 - Kombiniramo operacije
 - $\text{Cena} * (100 + \text{DDV}) / 100$

Izrazi z uporabo vrednosti

- *Delite skupno vrednost vseh računov s povprečno vrednostjo vseh računov.*
- *Izpišite nove vrednost računov, kjer od vsakega skupnega zneska odštejete 2€.*
- *Za vse račune strank, ki jim je ime Tina, skupen znesek razpolovite in ga zaokrožite na eno decimalko.*

Napredni izrazi z uporabo vrednosti

- Če želimo različno predstavitev podatkov glede na različne scenarije, znotraj programskih jezikov se uporabljajo stavki **SWITCH...CASE**.
- Določeni SUPB-ji podpirajo te možnosti, vendar v nekoliko drugačni obliki:
 - SWITCH...CASE je podprt v obliki ukaza **CASE**
- Velikokrat imamo NULL vrednosti, ki jih moramo manipulirati, kar omogočata ukaza: **NULLIF** in **COALESCE**

Ukaz CASE

- Stavke **CASE** pišemo znotraj SELECT dela in celoten CASE obravnavamo kot poseben stolpec:
 - Potrebno ga je **poimenovati**.
- Splošna sintaksa:

```
CASE
    WHEN iskalni-pogoj1 THEN rezultat1
    WHEN iskalni-pogoj2 THEN rezultat2
    WHEN iskalni-pogoj3 THEN rezultat3
    ...
    WHEN iskalni-pogojn THEN rezultatn
    [ELSE rezultatm]
END
```

- Če **ELSE** dela ne definiramo, je privzeto ELSE **null**.

Ukaz NULLIF

- Vrednost stolpca lahko **prikažemo** kot **NULL**, če je **enaka** določeni **preddefinirani vrednosti**.
- V oklepaju navedemo **stolpec**, ki ga **preverjamo** ter **vrednost**, ki jo želimo **zamenjati** z **NULL**.
- *Pri naslovih navedimo državo samo za stranke, ki ne prihajajo iz Slovenije.*

Ukaz COALESCE

- Izraz **COALESCE** je **obratna** operacija izraza NULLIF.
- Vrednost stolpca, ki je **NULL**, želimo **prikazati** kot določeno **preddefinirano vrednost**.
- V oklepaju navedemo **stolpec**, ki ga **preverjamo** ter **vrednost**, ki jo **želimo** zamenjati.
- *Neznane vrednosti podjetji zapišimo kot ,Fizična oseba‘*

Delo z več tabelami

Delo z več tabelami

- Do sedaj smo **povpraševali** po **podatkih ene tabele**.
 - Sicer smo si pomagali z omejenimi podpoizvedovanji (npr. z uporabo IN, EXISTS).
- Informacije, ki jih pridobimo so **pomanjkljive**.
 - Bistvo relacijskega podatkovnega modela je razdelitev podatkov v več tabel.
- Povpraševanje po več tabelah je mogoče z uporabo **operacij nad množicami** ali **združitev**.
 - **JOIN**

<https://www.postgresql.org/docs/18/tutorial-join.html>

Klasična notranja eksplicitna združitev

- **Klasična notranja združitev (INNER JOIN)** izbere **vse vrstice** iz **obeh tabel**, kjer je prišlo **do ujemanja** med stolpci (t.j. ključi), ki jih navedemo kot združitveni pogoj.
- Lahko si predstavljamo tudi kot **presek** med dvema množicama.
- Združujemo lahko **več tabel hkrati**, vendar je pri tem potrebno določiti združitvene pogoje za vse združitve.

Klasična notranja eksplicitna združitev

- Eksplicitna oblika notranjega združevanja predvideva **eksplicitno** uporabo ključnih besed (t.j. ukaza) **JOIN** ali **INNER JOIN** ter **ON**.
 - Prednost: preglednost, saj eksplicitno določimo, da gre v tem primeru za notranjo združitev dveh tabel, in sicer preko točno določenih stolpcev. Prav tako so **združevalni pogoji** na ta način eksplicitno ločeni od **splošnih pogojev** (npr. WHERE count > 3).
- Tabele največkrat **združujemo** preko **primarnih in tujih ključev** (t.j. primarni ključ ene tabele je tuji ključ v drugi), vendar takšen način ni nujen, kar pomeni, da lahko tabele združujemo tudi preko katerih koli drugih stolpcev.

Klasična notranja eksplicitna združitev

- **Križna** združitev
 - Je smiselna?
- **Naravna** združitev
 - **INNER** lahko izpustimo
 - Presek

Klasična notranja eksplicitna združitev

- Če obstaja **enak identifikator stolpcev** v obeh tabelah in želimo enega izpisati, potem je potrebno **eksplicitno** z **uporabo imena tabele določiti** kateri stolpec želimo izpisati.
- Imamo **naziv** v tabeli **STRANKA** in v tabeli **RAČUN**.
- Združevanje **več tabel** z izpisom določenih stolpcev
- *Katere izdelke in koliko so v okviru posameznega računa kupile stranke?*

Klasična notranja eksplicitna združitev

- **Preimenovanje tabel** med poizvedovanje in združevanjem
- Pri združevanju tabel pišemo **identifikatorje tabel** in nato **identifikatorje stolpcev** `tabela1.primarni_ključ = tabela2.tuji_ključ`, kar pa je lahko zamudno in nepregledno.
 - Tabelam lahko med pisanjem **določimo sinonime**. Nato se na tabele sklicujemo s pomočjo sinonimov, ki so praviloma krajše in preprostejše oblike identifikatorjev.

Klasična notranja eksplicitna združitev

- *Izpišimo imena in priimek strank ter datume nakupov v primerih, ko so stranke kupile več kot 3 primere posameznega izdelka.*
- *Kateri izdelki so se pojavili na računih z zneskom več kot 20€?*
- *Še kakšna ideja?*

Notranja implicitna združitev

- **Implicitna oblika notranjega** združevanja je vsebinsko enaka eksplicitni. Razlika je le v sintaksi.
 - PREDNOST: v FROM delu SQL stavka določimo **seznam** vseh **tabel**, ki jih naš ukaz potrebuje, kar omogoča hiter pregled nad tem katere tabele združujemo.

Notranja implicitna združitev

- *Izpišite izdelke, ki jih je kupila kupka po imenu Tina.*
- *Izpišite vse podatke računa za fizične osebe.*
- *Izpišite kaj in koliko je kupila Tina 28. 2. 2023.*
- *Izpišimo ime in priimek stranke, ki ima vsaj en račun.*
 - Se spomnimo?